Problem Statement: Building an RAG Application from a Scanned PDF Document

Objective

Create a **Retrieval-Augmented Generation (RAG)** application that processes a **scanned PDF document**, extracts text (using OCR if needed), and enables querying via **API calls**. The focus is on the backend functionality—no frontend/UI is required.

Requirements

- **Document Processing**
  - Accept a **scanned PDF** (may contain non-searchable text).
  - Use **OCR (Optical Character Recognition)** to extract text if needed (e.g., Tesseract, PyPDF2, or other tools).
  - Store the extracted text in a **vector database** (e.g., FAISS, Chroma, Pinecone, or Weaviate) for efficient retrieval.
- **Retrieval-Augmented Generation (RAG) Setup**
  - Split the extracted text into **chunks** (using LangChain, LlamaIndex, or custom logic).
  - Generate **embeddings** (using OpenAI, Hugging Face, or open-source models like all-MiniLM-L6-v2).
  - Implement a **retriever** to fetch relevant chunks based on user queries.
- **Query Handling via API**
  - Build a **FastAPI/Flask API** endpoint that:
    - Accepts a **user query** (via POST request).
    - Retrieves relevant text chunks from the vector DB.
    - Passes them to an **LLM** (GPT-4, Gemini, Llama 3, or any other model) to generate a response.
    - Returns the response in JSON format.
- **Optional (Good to Have)**
  - Support for **multiple PDFs** (batch processing).
  - Basic **query caching** to reduce LLM calls.
  - Logging for debugging.

Deliverables

- A working Python script or Jupyter notebook demonstrating:
  - PDF text extraction (OCR if needed).
  - Embedding generation & vector storage.
  - API endpoint for queries.
- A **Postman/curl example** to test the API.
- A brief **README** explaining setup and usage.

Evaluation Criteria

- **Functionality** – Does the API correctly process queries and return responses?
- **Code Structure** – Is the implementation modular and maintainable?
- **Choice of Tools** – Are the selected OCR, embedding, and LLM components appropriate?
- **Error Handling** – Does the code handle edge cases (e.g., empty PDFs, failed OCR)?

Tech Stack Suggestions

- **OCR:** PyPDF2, pdf2image + Tesseract, or unstructured.io.
- **Embeddings:** OpenAI text-embedding-3-small, Hugging Face sentence-transformers, or FastEmbed.
- **Vector DB:** FAISS (local), Chroma, or Pinecone (cloud).
- **LLM:** OpenAI GPT-4, Gemini, or open-source models (Llama 3, Mistral).

- **API:** FastAPI or Flask.

Example API Call

```
curl -X POST "http://localhost:8000/query" \
-H "Content-Type: application/json" \
-d '{"query": "What is the main topic of the document?"}'
```

Expected Response:

```
{
  "response": "The document discusses...",
  "relevant_chunks": ["...", "..."]
}
```

Note: Accuracy is not the primary focus, but the system should demonstrate a working RAG pipeline. We will discuss the implementation and potential improvements during the review. Would you like any refinements or additional constraints?

In the Given data/pdf document.
Few queries that can be tested:
Q. What is the CPT code for this case.
A. 64721,25447

Q. What is the diagnosis code for this case?
A. M18.11, G56.01

Q. List all previous medication for this patient?
A. rabeprazole 20, buspirone 15 ……..